MILK10k Classical Machine Learning Midterm

Computer Vision and Speech Recognition — Classical baseline

Author: Kacper Dudzik

Repository: https://github.com/KacperDudzik/CV_session

Project folder: ML_midterm_project/

1. Task

Develop, train and evaluate a classical ML baseline for skin-lesion classification on MILK10k. The pipeline compares feature sets and models under a lesion-grouped 70/20/10 split, selects by validation macro-F1, and scores the frozen winner once on an untouched test set.

2. How to run

From `ML_midterm_project/` with MILK10k available as `../milk10k` or `MILK10K_DIR`:

```
pip install -r requirements.txt
python main.py
```

Features are cached in `outputs/features_v2.csv`. Re-runs reuse the cache when the feature definition version matches.

3. Winning configuration

Model: SVM (RBF, C=1.0) · Feature set: `all_features` · Train balance: oversample · Selection: lesion-level macro-F1 on validation.

Split	Accuracy	Precision	Recall	F1	Satisfaction
Validation	—	0.552	0.593	0.567	0.571
Test	0.740	0.577	0.634	0.597	0.603
Majority (test)	—	0.232	0.333	0.273	0.279

Test ROC-AUC (macro OVR) 0.805 · PR-AUC 0.615 · malignant FN 89/364 · grouped CV macro-F1 0.484 ± 0.011 (train+val only).

4. Source code (GitHub)

Pipeline modules:

- main.py — Entry point: split, extract, search, CV, test once
- config.py — Paths, seed, split folds, feature-cache version, search grids
- data.py — Load MILK10k; lesion-grouped 70/20/10 split
- preprocess.py — Resize, RGB/luma/HSV/Lab, normalization
- features.py — Original Session-5 features + extended extract_all_features
- kernels.py — Box/Gaussian/sharpen/emboss/Sobel/Laplacian/Gabor + map summaries
- descriptors.py — Optional Canny, Harris, LBP, HOG, SIFT
- shape_texture.py — GLCM, colour clusters, heuristic mask shape
- augment.py — Train-only flips, rotation, brightness/contrast, safe crop
- balance.py — Oversample, undersample, targeted augmentation
- quality.py — Missingness, near-duplicates, leakage, class distributions
- train.py — Feature cache, model search, grouped CV
- evaluate.py — Lesion metrics, AUC, importance, REPORT artifacts
- requirements.txt — Python dependencies

Models (one file each):

- models/majority.py — Majority-class baseline
- models/logistic.py — Logistic regression
- models/tree.py — Decision tree
- models/knn.py — k-nearest neighbours
- models/svm.py — SVM linear / RBF (calibrated)
- models/forest.py — Random forest
- models/__init__.py — Candidate registry

5. Results and artifacts (GitHub)

All metrics below were produced by the pipeline; nothing is fabricated. Click through to the corresponding file in the repository.

- outputs/REPORT.md — Methods and results narrative
- outputs/test_metrics.json — Frozen test metrics
- outputs/best_config.json — Winning configuration
- outputs/val_leaderboard.csv — Full validation leaderboard
- outputs/feature_ablation.csv — Feature-set ablation
- outputs/cv_folds.csv — Grouped CV fold scores
- outputs/confusion_test.csv — Test confusion matrix
- outputs/confusion_test.png — Confusion figure
- outputs/feature_importance.csv — Feature importance table
- outputs/feature_importance.png — Importance figure
- outputs/class_distribution.png — Class-balance figure
- outputs/class_balance_report.csv — Balance before/after
- outputs/subgroup_metrics.csv — Subgroup scores
- outputs/train_lesions.csv — Train lesion IDs
- outputs/val_lesions.csv — Validation lesion IDs
- outputs/test_lesions.csv — Test lesion IDs
- outputs/model.joblib — Fitted winning model
- outputs/features_v2.csv — Cached feature matrix

6. Design notes

Lesion identity is never split across train/val/test. Both dermoscopic and clinical views belong to the same split; class probabilities are averaged before the lesion label is chosen. Augmentation and resampling are train-only. The test set is unused until the configuration is frozen. Optional descriptors (Canny, Harris, LBP, HOG, SIFT) are clearly labeled in code. Physical diameter and ABCDE evolution are not claimed without calibration or longitudinal images.